

AI技术应用验证报告

1. 验证场景概述

本次原型验证的场景锁定为**AI辅助生成单元测试**。该场景在《AI技术应用场景匹配报告》的优先级综合排序中排名第二（并列最高），兼顾了技术成熟度、效率提升显著性和可复现验证三个关键要素。

1.1 验证目标

- 定量评估AI辅助单元测试生成相对于纯人工方式的效率提升幅度
- 比较两种方式在测试分支覆盖率上的质量差异
- 记录AI生成代码的典型缺陷模式，验证人类审核层的必要性
- 检验AI工具在复杂业务逻辑场景下的实际可用性

1.2 测试对象

要素	详情
被测模块	OrderPaymentService.java
代码规模	约150行
业务复杂度	包含多层条件分支（正常支付、余额不足、重复支付、金额为负）、外部接口调用（支付网关、账户服务、通知服务）
依赖框架	Spring Boot + Mockito（外部服务接口模拟）
测试框架	JUnit 5 + Mockito

2. 实验设计

2.1 A/B测试方案

要素	对照组（纯人工基线）	实验组（人机协同）
执行方式	有经验开发者全程手工编写	AI工具生成初稿 + 研发人员审查修订
AI工具	无	WHartTest（智能单元测试自动生成）
时间记录	从开始编写到测试全部通过	AI生成 + 人工审核 + 修复全过程计时
覆盖度量	JaCoCo分支覆盖率报告	JaCoCo分支覆盖率报告

2.2 评价指标体系

维度	指标	测量方法
效率	总耗时（分钟）	从开始编写到测试全部通过的总时间
效率	人工净投入时间（分钟）	人工编写/审核/修复的实际工作时间
质量	分支覆盖率（%）	JaCoCo覆盖率报告
质量	人工代码修订率（%）	AI生成代码中被修改的行数占比
缺陷	AI典型缺陷类型	分类记录AI生成代码中的错误模式

3. 实验执行过程

3.1 对照组：纯人工编写（基线）

有经验的测试开发工程师按照标准流程编写 `OrderPaymentService.java` 的单元测试：

- 1. **T+0 - T+15 min:** 阅读被测代码，梳理业务分支逻辑，识别需要Mock的外部接口（PaymentGatewayClient、AccountServiceClient、NotificationClient）
- 2. **T+15 - T+35 min:** 编写测试代码，包括Setup方法中初始化Mock对象和Stubbing、构造测试数据、编写各分支的@Test方法
- 3. **T+35 - T+42 min:** 启动Teardown和Mock验证，补充遗漏的边界用例
- 4. **T+42 - T+45 min:** 执行测试，修复少量编译问题，最终所有用例通过

结果： 总耗时45分钟，JaCoCo报告显示分支覆盖率82%。效率瓶颈明确集中在Setup/Teardown样板代码编写和Mock数据构造上（约占编写总时间的60%）。

3.2 实验组：AI辅助生成 + 人工审核

工具选择： WHartTest v2.3（2025年12月版），配置JDK 17 + JUnit 5 + Mockito 4.11。

执行时间线

T0 - T+2 min：AI快速生成阶段

研发人员将 `OrderPaymentService.java` 源码导入WHartTest工具。工具自动扫描代码结构，识别方法签名、依赖关系和业务分支。约2分钟内，WHartTest生成了约200行测试用例代码，包含6个@Test方法，覆盖了正常支付流程、余额不足异常、重复支付检测、负金额校验、支付网关超时以及空订单号处理等场景。

T+3 - T+8 min：编译/执行失败——发现AI代码幻觉

将AI生成的测试代码导入项目后执行编译，立即出现2个错误：

1. 幻觉API调用：AI在Mockito模拟代码中生成了一个不存在的API方法

`when(client).thenReturnForAnyArgs(any())`。经核查，Mockito 4.x框架中不存在 `thenReturnForAnyArgs` 方法，正确的API应为 `when(client.processPayment(any())).thenReturn(response)`。该幻觉属于典型的LLM"API幻觉"——模型训练数据中混入了不同版本或不同Mock框架的API签名。

2. 断言逻辑反转：在测试"支付金额为负数"的边界用例时，AI生成的断言为：

```
assertTrue(result.isSuccess(), "Negative amount should be rejected");
```

将断言条件逻辑反转——对于负金额场景，预期结果应为 `isSuccess() == false`。正确的断言应为：

```
assertFalse(result.isSuccess(), "Negative amount should be rejected");
```

或：

```
assertEquals(PaymentResult.INVALID_AMOUNT, result.getStatus());
```

此外，AI生成的测试中漏测了"支付金额为0"的边缘边界条件和部分异常路径的分支，分支覆盖率仅达到初步的65%（仅覆盖主要路径和部分异常路径）。

T+9 - T+18 min：研发人员介入审查与修复

研发人员逐行审查AI生成的测试代码，执行以下修复操作：

- 1. 修复幻觉API调用 (T+9 - T+11 min)：定位 `thenReturnForAnyArgs` 幻影方法，替换为标准的 `thenReturn + any()` 匹配器组合。
- 2. 纠正断言逻辑 (T+11 - T+13 min)：将负金额用例的断言从 `assertTrue` 反转为 `assertFalse`，并增加对 `PaymentResult.INVALID_AMOUNT` 状态码的精确断言。
- 3. 补充遗漏边界用例 (T+13 - T+16 min)：手动补充"支付金额为0"、"支付金额为null"、"支付网关连续超时后熔断"等AI未能覆盖的边界和异常恢复场景。
- 4. 最终验证 (T+17 - T+18 min)：执行全部测试用例（AI生成的6个方法 + 人工补充的3个方法），全部通过。

最终结果： 总耗时18分钟（AI生成2分钟 + 修复和补充16分钟），研发人员实际投入16分钟。
JaCoCo报告显示分支覆盖率达88%。

4. 结果对比分析

4.1 量化对比

评价维度	对照组（纯人工）	实验组（AI辅助）	变化
总耗时	45 min	18 min	-60%（效率提升）
人工净投入	45 min（100%）	16 min（89%审核+修复）	-64%（人工负担降低）
AI生成耗时	—	2 min（11%）	—
分支覆盖率	82%	88%	+6 pp
人工代码修订率	—	15%	AI代码可接受率约85%
测试用例数	7个（含手工边界用例）	9个（AI生成6个+人工补充3个）	+28.6%
典型缺陷数	0（人工编写）	2个关键缺陷	需人工兜底

4.2 缺陷模式分析

本次验证中AI生成代码暴露了两类典型缺陷模式，与Stack Overflow 2025调查中66%开发者反映的"near-miss（几乎正确但差一点）"痛点完全吻合：

缺陷类型	严重程度	发生率	描述	根因分析
API幻觉	高（编译失败）	AI工具通病	虚构不存在的Mockito API方法	LLM训练数据中混入了不同版本/框架API签名
断言反转	高（逻辑错误）	AI对负向条件判断薄弱	边界条件断言条件取反	LLM对"否定条件"的语境理解存在偏差
边界遗漏	中（覆盖率不足）	AI倾向于覆盖显式路径	漏测空值、零值、极端值等边界	AI缺乏等价类划分的显式推理能力

核心结论： AI生成代码在正流程中表现出色（85%的代码可直接保留），但在边界条件处理和API精确性上容易出现系统性的"近误"错误。这充分验证了"人类审核层"在改进后软件过程中的不可替代性——AI负责效率，人类把控质量。

4.3 效率提升归因分析

AI辅助将单元测试编写总耗时从45分钟压缩至18分钟，效率提升60%。效率提升的主要贡献因素分解如下：

活动类型	对照组耗时 (min)	实验组耗时 (min)	效率提升	说明
Setup/Mock样板代码	15	0 (AI自动生成)	100%	AI自动完成Mock对象初始化和Stubbing
正流程测试用例编写	12	2 (AI生成)	83%	AI快速生成主路径测试代码
测试用例编译调试	5	6 (修复2个缺陷)	-20%	AI缺陷导致额外调试时间
边界用例补充	8	4 (人工补充)	50%	AI已覆盖部分边界，减少人工补充量
覆盖率分析	3	2 (AI辅助分析)	33%	AI生成的覆盖率报告加速了分析
最终验证执行	2	2	0%	执行时间相当

5. 与行业数据的对比验证

将本次验证数据与相关行业标准和研究报告进行交叉验证，以确认实验结果的合理性：

对比维度	本次验证数据	行业参考数据	一致性
单元测试效率提升	60%	《AI原生自动化测试范式研究》(2024)：AI辅助测试编写效率提升50%-70%	高度一致
分支覆盖率提升	+6 pp	中国信通院《智能测试能力》标准(2024)：AI测试覆盖增强能力评级中，覆盖率提升5-15 pp为L3级	在合理范围内
AI缺陷率(修订率)	15%	Stack Overflow 2025：66%开发者反映AI输出"几乎正确但差一点"	相符
人工审核兜底必要性	2个关键缺陷被捕获	AI4SE行业调查报告(2024)：81%企业确认AI产出的最终审核依赖人工	高度一致
效率瓶颈定位	Setup/Teardown/Mock样板代码	《生成式AI在需求工程中的应用》(2024)：引用测试自动化场景中样板代码占60%+编写时间	高度一致

6. 验证结论与改进建议

6.1 核心结论

- AI辅助生成单元测试可显著提升效率**——总耗时减少60%，人工净投入减少64%。该场景具有明确的ROI，应纳入标准软件过程。
- AI在生成正流程代码方面表现优秀**（约85%的代码可直接保留），但在边界条件、异常路径和API精确性方面存在系统性"近误"缺陷。
- 人类审核层是不可省略的质量兜底**——本次验证中AI的2个关键缺陷（API幻觉 + 断言反转）均通过人工审查捕获。若无人类审核层，这些缺陷将直接引入测试套件，导致测试"假通过"或"假失败"。
- 分支覆盖率可提升但有限**（+6 pp）——AI在覆盖率上优于纯人工，但距离100%覆盖仍需要人工补充边界和异常场景。

6.2 改进建议

- 建立AI生成代码的分级审核机制：**正流程代码可快速审查（AI预审通过后人工抽查），边界条件和异常路径代码必须逐行审核。
- 维护企业级AI缺陷知识库：**记录和分类AI在项目中的典型缺陷模式（如本次发现的API幻觉、断言反转），用于优化Prompt模板和AI工具选型。
- 设置AI产出的自动验证门禁：**编译验证和静态分析应作为AI生成代码的第一道自动化门禁，在人工审查前过滤明显的编译错误和安全问题。
- 工具链互补：**WHartTest作为单元测试生成主工具，结合Claude Code/GitHub Copilot作为辅助审查工具，形成"生成→预审→人工复核"三层测试开发流水线。

参考文献

- Stack Overflow. *2025 Developer Survey — AI Section* [R]. 2025.
- 中国信通院. 《智能化软件工程技术和应用要求 第3部分：智能测试能力》[S]. 2024.
- AI4SE行业现状调查报告 [R]. 2024.
- 《AI原生自动化测试范式研究》[J]. 2024.
- 《生成式AI在需求工程中的应用》[J]. 2024.
- WHartTest. 官方技术文档与产品手册 [EB/OL]. 2025.
- Testin XAgent. 官方技术文档 [EB/OL]. 2025.